

Pokémon TCG Explorer

Documentación técnica del proyecto

PRUEBA TÉCNICA FRONT-END · REACT

Entrega de la prueba técnica Front-End React, adaptada de la temática original de Marvel API a Pokémon TCG API y PokéAPI, con autorización previa del cambio.

Julio 2026

1. Descripción general

Pokémon TCG Explorer es una aplicación Front-End construida con React 19 y Vite que permite buscar, listar y consultar el detalle de cartas del Pokémon Trading Card Game y de Pokémon de la Pokédex. Consume dos APIs externas de forma directa desde el navegador: la Pokémon TCG API, que requiere una API key, y PokéAPI, que es pública y no requiere autenticación.

El proyecto originalmente pedía integración con Marvel API, cambio autorizado por el solicitante de la prueba hacia una temática de Pokémon. Posteriormente se recibió un segundo documento de requerimientos centrado en PokéAPI, cuyos puntos de historial de búsquedas (persistencia en base de datos y gestión de usuarios) quedaron fuera de alcance de forma explícita y acordada, dado que el proyecto no incluye backend propio ni base de datos.

2. Funcionalidad implementada

- Buscador único, disponible en el Navbar, en el Hero de la página de inicio y dentro de cada listado, que identifica automáticamente si el texto ingresado corresponde a un nombre, un tipo o un código exacto.
- Los buscadores del Navbar y del Hero, al no estar ligados a una sección fija, preguntan mediante un modal si el término se quiere buscar entre las cartas del TCG o en la Pokédex, antes de navegar al resultado.
- Búsqueda en vivo dentro de los listados de cartas y de Pokémon, los resultados se actualizan automáticamente mientras se escribe, con una pausa breve (debounce de 550ms) para no disparar una petición por cada tecla.
- Listado de cartas con paginación real, filtrable por nombre o tipo.
- Detalle de cada carta con imagen, descripción, HP, rareza, set, artista, y las demás cartas del mismo set como contenido relacionado.
- Funcionalidad equivalente para Pokémon, listado paginado y detalle con estadísticas, habilidades y movimientos.
- Página de inicio con accesos rápidos por tipo, que permite elegir entre ver cartas o Pokémon de ese tipo, además de una vista previa de ambos catálogos.
- Manejo de estados de carga, error y resultados vacíos en todos los listados y detalles, con reintento manual y recuperación automática ante fallos de red puntuales.
- Diseño responsivo e interfaz animada en transiciones, listados y modales.
- Página 404 para rutas no reconocidas.

Los requerimientos de historial de búsquedas (puntos 6 y 7 del documento original de Marvel, y del 6 al 10 del documento de PokéAPI recibido posteriormente) quedaron fuera de alcance de forma explícita, dado que no aplican sin base de datos ni gestión de usuarios, según lo acordado con quien encargó la prueba.

3. Arquitectura y estructura del proyecto

La aplicación es exclusivamente Front-End, sin backend propio. Ambas APIs se consumen directamente desde el navegador. No se utiliza TypeScript, en su lugar los tipos de datos se documentan con JSDoc en `models/`. Tampoco se utiliza Redux ni una librería de estado global, cada página maneja su propio estado mediante hooks, dado que no existe información que deba compartirse entre rutas de forma que lo justifique.

```

src/
  components/
    layout/      Navbar, Footer, Hero
    ui/          Loader, ErrorMessage, EmptyState, Pagination, TypeIcon, Modal
    cards/       PokemonCard, CardGrid, CardDetail
    pokemon/     PokemonCreatureCard, PokemonGrid, PokemonDetail
    search/      SearchBar (componente único, reutilizado con el prop mode
                  y soporte de búsqueda en vivo vía el prop live),
                  SearchModeModal (modal compartido para elegir entre
                  cartas y Pokédex)

  pages/
    Home, Cards, CardDetail, Pokemon, PokemonDetail, NotFound
  hooks/
    useCards, useCardDetail, useRelatedCards,
    usePokemonList, usePokemonDetail, useSearchModeChoice
  services/     pokemon.service.js, pokeapi.service.js, httpClient.js
  models/       tipos documentados con JSDoc (PokemonCard.js,
                  SearchFilters.js)
  routes/       AppRoutes.jsx
  styles/       variables.css, global.css
  utils/        constants.js, formatters.js, search.js, requestCache.js

```

3.1 Componente de búsqueda

`SearchBar` es un único componente reutilizado en el Navbar, el Hero de la página de inicio y dentro de los listados de cartas y Pokémon. Recibe un prop `mode` (`cards` o `pokemon`) que determina a qué API apunta y qué lógica de detección de intención usa, y un prop `live` que activa la búsqueda en vivo con debounce mediante `navigate(..., { replace: true })` sobre la ruta actual.

En el Navbar y en el Hero, donde el buscador no está ligado a una sección fija, el prop `onRequireMode` reemplaza la navegación directa, delegando la decisión a un hook compartido (`useSearchModeChoice`) que abre el modal `SearchModeModal` y solo navega una vez que se elige cartas o Pokédex.

4. Rutas

Ruta	Contenido
<code>/</code>	Inicio, buscador, categorías por tipo, cartas y Pokémon destacados
<code>/cards</code>	Listado de cartas, con filtros y paginación
<code>/cards/:id</code>	Detalle de una carta
<code>/pokemon</code>	Listado de Pokémon, con filtros y paginación
<code>/pokemon/:idOrName</code>	Detalle de un Pokémon
Cualquier otra	Página 404

5. Integración con las APIs

5.1 Pokémon TCG API

Se consume directamente desde `src/services/pokemon.service.js` contra `https://api.pokemontcg.io/v2`, enviando la API key en el header `X-API-Key`. Las búsquedas por nombre, tipo y set se traducen a consultas con sintaxis Lucene propia de esta API, por ejemplo `name:"pikachu*"` o `types:Fire`.

5.2 PokéAPI

Se consume directamente desde `src/services/pokeapi.service.js` contra `https://pokeapi.co/api/v2`, sin necesidad de key. La búsqueda parcial por nombre requería apoyarse en información persistida o sincronizada, y al no incluir base de datos, se resolvió con una caché en memoria del lado del cliente, en el primer uso se obtiene el listado completo de nombres de PokéAPI (aproximadamente 1300) y se mantiene en una variable del módulo mientras dure la sesión del navegador.

Los tipos no coinciden entre ambas APIs, por ejemplo la TCG usa `Lightning` mientras que PokéAPI usa `electric`, por lo que se implementó un mapeo en `utils/constants.js` para que el buscador reconozca ambos vocabularios, en español e inglés.

6. Rendimiento y resiliencia

Al consumir ambas APIs directamente desde el navegador, sin un proxy propio que amortigüe la latencia o los fallos de red, se agregaron varias capas de resiliencia:

- **Caché de peticiones** (`utils/requestCache.js`): cada consulta a la API queda guardada en memoria por su combinación exacta de parámetros, si se repite la misma consulta durante la sesión del navegador, se devuelve al instante sin volver a golpear la red.
- **Reintento automático**: si una petición falla por un error de red puntual (no por una respuesta de error de la API), se reintenta una vez antes de mostrar el mensaje de error al usuario.
- **Descarte de respuestas fuera de orden**: en los listados con búsqueda en vivo, cada petición lleva un identificador incremental, si una respuesta antigua llega después de una más reciente, se descarta para no pisar el resultado actual ni mostrar un conteo de resultados inconsistente con lo que se ve en pantalla.
- **Modales sin `backdrop-filter`**: se retiró el desenfoque de fondo de los modales, ya que recalcularlo en cada frame junto con las animaciones continuas del Hero generaba una degradación perceptible al abrirlas.

7. Nota sobre la API key

La Pokémon TCG API requiere una key enviada en el header `X-API-Key`. Se evaluó agregar un backend propio que actuara como proxy para mantener la key fuera del bundle público, pero en un proyecto sin base de datos ni infraestructura propia ese proxy solo agrega un proceso extra para desplegar y mantener, sin aportar una barrera real contra alguien decidido a extraerla. Se optó por una alternativa más simple, la key vive en el `.env` del proyecto y viaja en el JavaScript del build final, igual que ocurriría con cualquier variable `VITE_*`.

Esto significa que cualquiera que inspeccione el bundle publicado puede ver la key, es una limitación conocida y aceptada para el alcance de esta entrega, no un descuido. En un proyecto real con backend propio, la key

volvería a vivir del lado del servidor.

8. Variables de entorno

En la raíz del proyecto, archivo `.env`:

```
VITE_POKEMON_API_KEY=
```

La key de Pokémon TCG API se obtiene sin costo en pokemontcg.io/dashboard.

9. Instalación y ejecución local

Requisitos: Node.js 20 o superior y npm.

```
npm install
cp .env.example .env      # completar VITE_POKEMON_API_KEY
npm run dev                # queda disponible en localhost:5173
```

La key de PokéAPI no es necesaria, dado que es una API abierta. Si no se dispone de una key de Pokémon TCG API, la aplicación funciona igual, aunque con el límite gratuito compartido, que es considerablemente más bajo.

10. Build de producción

```
npm run build
npm run preview
```

11. Pruebas realizadas

El proyecto no cuenta con una suite de pruebas automatizadas, la verificación se hizo de forma manual antes de cada entrega:

- **Lint:** `npm run lint` corre limpio, 0 errores, con 9 advertencias intencionales de `react-hooks/exhaustive-deps`.
- **Build:** `npm run build` y `npm run preview` verificados sin errores.
- **Funcional, manual:** navegación entre rutas, búsqueda por nombre, tipo y código exacto, búsqueda en vivo, modal de selección de sección desde ambos buscadores, paginación, detalle de carta con relacionadas, detalle de Pokémon, y estados de carga, error y vacío.
- **Responsivo:** escritorio, tablet y mobile, incluyendo la corrección de un desbordamiento horizontal por el comportamiento por defecto de CSS Grid.
- **Integración directa con las APIs:** verificado con peticiones directas que ambas APIs responden correctamente desde el navegador sin bloqueos de CORS.
- **Exposición de la API key:** confirmado que solo aparece en `dist/` de forma intencional, y nunca fuera del `.env` gitignoreado.

12. Despliegue

El Front-End se publica en GitHub Pages mediante GitHub Actions, sin necesidad de backend propio ni de un servicio de hosting adicional.

GitHub Pages sirve el sitio desde una subcarpeta (`sglmz.github.io/pokemonPruebaAPI/`), por lo que el build necesita conocer ese subpath de antemano. El workflow de GitHub Actions (`.github/workflows/deploy.yml`) lo resuelve automáticamente mediante la variable `VITE_BASE_PATH` , definida únicamente en ese workflow, y toma la key desde el secret `VITE_POKEMON_API_KEY` configurado en el repositorio. También copia `index.html` a `404.html` tras el build para que las rutas de React Router funcionen al recargar la página o entrar por un enlace directo a una ruta interna (por ejemplo `/cards/base1-4`).

El despliegue se dispara automáticamente con cada `push` a la rama `main` , y también puede ejecutarse manualmente desde la pestaña Actions del repositorio.

13. Decisiones técnicas

- No se utilizó TypeScript, se documentaron los tipos de datos con JSDoc en `models/` , manteniendo el proyecto en JavaScript puro.
- No se utilizó Redux ni una librería de estado global equivalente, cada página maneja su propio estado mediante hooks.
- No se implementó un backend propio, ambas APIs se consumen directamente desde el cliente, según lo justificado en la sección 7.
- La búsqueda en vivo usa `navigate` con `replace: true` en lugar de `push` , para no llenar el historial de navegación con una entrada por cada letra escrita.
- La lógica de elegir entre cartas y Pokédex se extrajo a un hook (`useSearchModeChoice`) y a un componente de modal compartido (`SearchModeModal`), reutilizado por el Navbar y el Hero en lugar de duplicarse.

14. Pendientes y alcance no cubierto

- El proyecto no cuenta todavía con pruebas automatizadas.
- Historial global de búsquedas y por usuario, fuera de alcance según lo acordado, por no incluir base de datos ni gestión de usuarios.